

Technical Report: Explanations of Slides Graph-Based Error Correcting Code (Fall 2025)

Hassan Tavakoli
Oregon State University

June 11, 2026

Contents

Title / first page	3
Agenda	4
Motivation to Problem of Information Transmission	5
Redundancy as Protection	6
Example: Communicating the Word “red” (first)	7
Example: Communicating the Word “red” (second)	8
The communication problem	9
Binary Symmetric Channel (BSC) — notation	10
single-bit+one-copy (repetition code)	11
MAP decoding (repetition), case $q > 1/2$	12
MAP decoding (repetition), case $q < 1/2$	13
Average error probability (MAP)	14
Graphical Thinking (first)	15
Graphical Thinking (second)	16
Graphical encoding for Hamming(7,4)	17
Graph-Based-Codes-1	18
Graph-Based-Codes-2	19
Graph-Based-Codes-3	20

What is a factor graph?	21
Example factor graph	22
More Complex Factor Graph with joint factors	23
Message Passing	24
Belief Propagation (BP)	25
Message Passing on a Factor Graph (SP Algorithm)	26
Example: Graph structure for Assumptions	27
Case A — Original priors (part 1)	28
Case A — Original priors (part 2)	29
Case B — Adjusted priors (part 1)	30
Case B — Adjusted priors (part 2)	31
Comparison and interpretation	32
Optional: threshold condition	33
Factor Graph structure for Graph-Based Code	34
Binary Erasure Channel and Peeling decoder (intro)	35
Binary Erasure Channel and Peeling decoder (table)	36
Peeling decoder example (1)	37
Peeling decoder (decoding step)	38
Peeling decoder example (2)	39
Peeling decoder example (3)	40
LDPC Degree Distributions on BEC	41
Density Evolution-1	42
Density Evolution-2	43
Optimal Rate (SIP) - 1	44
Optimal Rate (SIP) - 2	45
Optimal Rate (SIP) - 3	46

Slide: Title / first page

Graph-Based Error Correcting Code

Hassan Tavakoli

Fall 2025 — Oregon State University



Oregon State
University



Navigation icons: back, forward, search, etc.

Hassan Tavakoli Graph-Based Error Correcting Code 1 / 44

Slide: Agenda

Agenda

- 1 Title & goals
- 2 Motivation: noise in communication
- 3 Redundancy & simple codes (repetition example)
- 4 Communication model: encoder, channel (BSC), decoder
- 5 MAP decoding results for repetition/parity
- 6 Graphical view: factor/Tanner graphs
- 7 Hamming(7,4) — graphical encoding and numeric example
- 8 Graph-based codes: LDPC, rate, sparsity, complexity
- 9 Message passing & Belief Propagation (sum/max rules)
- 10 BEC & Peeling decoder (worked bipartite example)
- 11 Density evolution: recursion and threshold intuition
- 12 Design problem: optimal irregular LDPC rate (semi-infinite program)
- 13 Takeaways, limitations, next steps
- 14 Q&A



Hassan Tavakoli Graph-Based Error Correcting Code 2 / 44

The agenda slide lists the talk's structure in order: motivation, simple codes, channel model, MAP analysis, graphical models, Hamming example, LDPC and decoding, BEC/peeling, density evolution, design as a semi-infinite optimization, and final takeaways. Treat this as a roadmap: each following section of the paper corresponds to a cluster of slides in the agenda. When presenting, call out which topics are background and which are the speaker's original contributions or main emphasis (e.g., the semi-infinite design problem).

Slide: Motivation to Problem of Information Transmission

Motivation to Problem of Information Transmission

- Information theory began as a **theory of communication**.
- In the late 1940s, **Claude Shannon**, an engineer at Bell Labs, focused on the problem of **noise** in communication systems.
- Even small imperfections in transmission lines can **distort signals** and corrupt data.
- How can we **protect messages** against noise?
- One obvious solution: use **better equipment** — higher-quality cables, amplifiers, and transmitters.
- But there's a **cheaper and smarter alternative**.

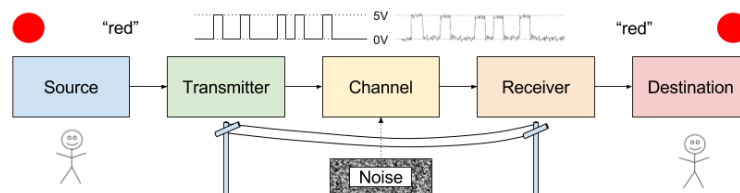


Figure: A typical transmission system.

Hassan Tavakoli

Graph-Based Error Correcting Code

3 / 44

This slide recaps the historical and conceptual motivation: Shannon's formulation of communication with noise and the central questions about reliability versus rate. The key point to stress is that improving hardware is not always the most cost-effective way to combat noise; coding (adding redundancy intelligently) can yield orders-of-magnitude improvements in reliability. This explains that this motivates both theoretical tools (capacity, error exponents) and practical coding strategies (redundancy, efficient decoding).

Slide: Redundancy as Protection

Redundancy as Protection

- Instead of improving hardware, we can **add redundancy** to our signals. Redundancy enables **error detection and correction** even when noise alters part of the message.
- However, redundancy isn't free — it **reduces the communication rate**. Shannon aimed to **quantify the trade-off** between reliability and rate.
- Is **error-free communication** possible over a noisy channel?
- If yes, **how much redundancy** (extra bits) does it require?
- To understand Shannon's insight, we first explore **basic error-correcting codes (ECCs)**. Start with the simplest example: **replication codes**. Each bit is sent multiple times to improve reliability —
- Example of **replication codes** $0 \rightarrow 000, 1 \rightarrow 111$.

This slide introduces the basic notion that deliberate redundancy enables error detection and correction, but at a cost to rate. Make explicit the trade-off Shannon formalized: reliable communication up to capacity is possible but requires sufficient redundancy (code rate less than channel capacity). We define rate R , redundancy, and explain the intuition that codes attempt to move the operating point below the channel's error threshold using minimal extra bits.

Slide: Example: Communicating the Word “red” (first)

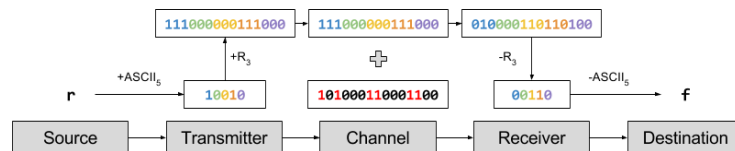
Example: Communicating the Word “red”

- Suppose I want to communicate the word **“red”** to you. Each letter is encoded into **ASCII bits** and sent over the Internet. Using a simplified ASCII: ‘r’ $\rightarrow$ 011 – 10010
- To protect against noise, each bit is repeated three times:

10010 $\rightarrow$ 111000000111000

Noise corruption: 111 – 000 – 000 – 111 – 000 $\Rightarrow$ 101 – 000 – 110 – 001 – 100

- The last error changes 000 $\rightarrow$ 100, which is ignored by the decoder.
- The first two noise events flip 111 $\rightarrow$ 010, causing a decoding error.
- After decoding, the ASCII interpretation changes — we will receive **“ted, 011-10100”** instead of **“red,011-10010”**.



This slide gives a concrete toy example: ASCII representation, bit-repetition (3×) as a replication code, and a demonstration of how a few flips can change the interpreted message. The pedagogical purpose is to show that naive replication reduces but does not eliminate errors — burst or correlated flips and multiple flips can still produce incorrect decoded symbols. We explain the decoding rule (majority vote for triple repetition) and the kinds of error patterns that defeat it, motivating more powerful codes than naive repetition.

Slide: Example: Communicating the Word “red” (second)

Example: Communicating the Word “red”

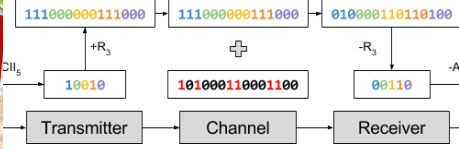
- Suppose I want to communicate the word “**red**” to you. Each letter is encoded into **ASCII bits** and sent over the Internet. Using a simplified ASCII: ‘r’ $\rightarrow$ 011 – 10010
- To protect against noise, each bit is repeated three times:

$$10010 \rightarrow 111000000111000$$

Noise corruption: 111 – 000 – 000 – 111 – 000 $\Rightarrow$ 101 – 000 – 110 – 001 – 100

- The last error changes 000 $\rightarrow$ 100, which is ignored by the decoder.
- The first two noise events flip 111 $\rightarrow$ 010, causing a decoding error.
- After decoding, the ASCII interpretation changes — we will receive “**ted, 011-10100**” instead of “**red,011-10010**”.

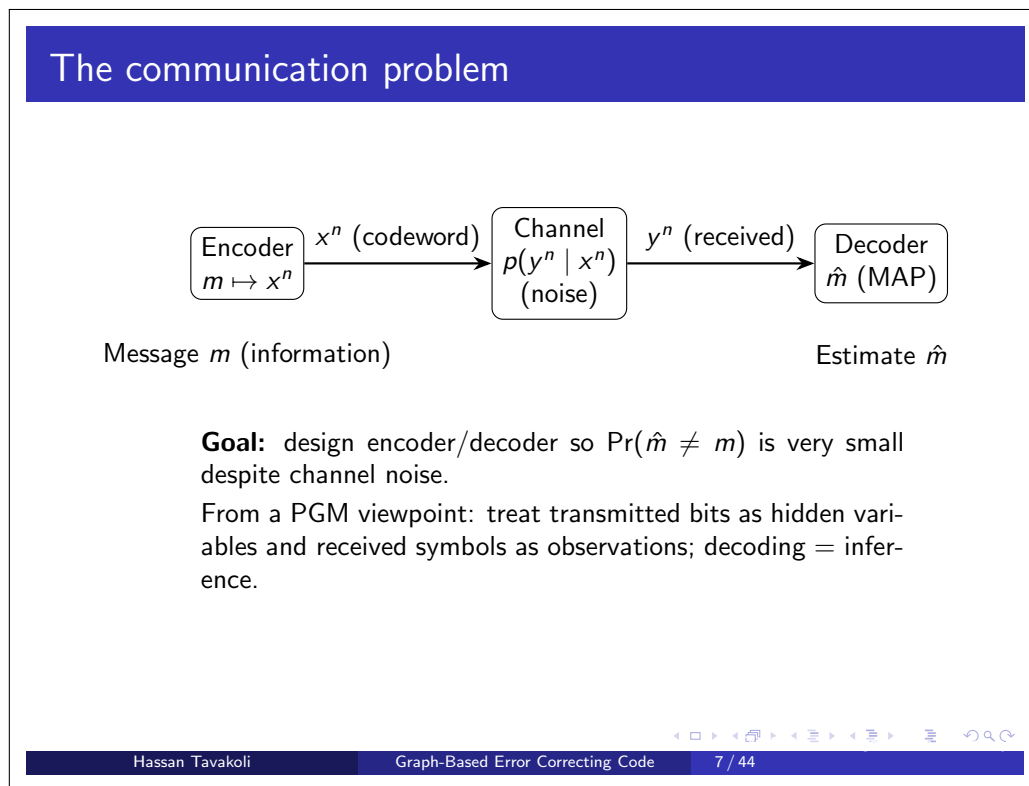


Hassan Tavakoli
Graph-Based Error Correcting Code
6 / 44

This repeated slide reinforces the same example with additional visuals; treat it as emphasis. We show how to compute probability of decoding error under an i.i.d. binary symmetric channel (BSC) model for repetition, and contrast that with the cost in rate (two extra bits per info bit for 3 $\times$ repetition). This slide is a natural place to introduce the BSC formally in the next slides.

Slide: The communication problem



This slide diagrams encoder→channel→decoder and frames decoding as probabilistic inference (MAP). We formalize the problem statement: given encoder $m \mapsto x^n$, channel $p(y^n | x^n)$, design decoder $\hat{m}(y^n)$ such that $\Pr(\hat{m} \neq m)$ is small. Emphasize that many decoding algorithms can be viewed as approximate inference under a graphical model representing the code and channel. Also note complexity constraints: theoretical optimality (ML/MAP) is often combinatorial, driving interest in efficient approximate algorithms (BP, peeling).

Slide: single-bit+one-copy (repetition code)

single-bit+one-copy (repetition code): send $(X, f(X))$,
receive two-bit Y with independent BSC flips.

X	$f(X)$	$\theta_{Xf(X)}$	X	$f(X)$	Y	$\theta_{Y Xf(X)}$
0	0	$1 - q$	0	0	00	$(1 - p)^2$
0	0		0	0	01	$p(1 - p) \rightarrow \hat{X} = ?$
0	0		0	0	10	$p(1 - p) \rightarrow \hat{X} = ?$
0	0		0	0	11	$p^2 \rightarrow \hat{X} = ?$
1	1	q	1	1	00	$p^2 \rightarrow \hat{X} = ?$
1	1		1	1	01	$p(1 - p) \rightarrow \hat{X} = ?$
1	1		1	1	10	$p(1 - p) \rightarrow \hat{X} = ?$
1	1		1	1	11	$(1 - p)^2$

$$P_{a_{ave}}^{(e)} = \frac{\Pr(Y \neq Xf(X))}{2} = \frac{1}{2}(1 - q)(2p(1 - p) + p^2) + \frac{1}{2}q(2p(1 - p) + p^2) \\ = p - \frac{p^2}{2}.$$

Navigation icons: back, forward, search, etc.

This slide computes joint probabilities for a simple repetition scheme (send X and one copy $f(X)$). We walk through the table of probabilities for all received patterns under independent BSC flips and show how the average error probability for a naive decoding rule is derived. This slide is a useful demonstration of how to compute exact error probabilities for tiny codes and to compare before/after redundancy.

Slide: MAP decoding (repetition), case $q > 1/2$

MAP decoding : single-bit+one-copy (repetition code):
send $(X, f(X))$, receive two-bit Y with independent BSC
flips.

Assume that $q > 1/2$

X	$f(X)$	$\theta_{Xf(X)}$	X	$f(X)$	Y	$\theta_{Y Xf(X)}$
0	0	$1-q$	0	0	00	$(1-p)^2 \rightarrow \hat{X} = 0$
0	0	$1-q$	0	0	01	$p(1-p) \rightarrow \hat{X} = 1$
0	0	$1-q$	0	0	10	$p(1-p) \rightarrow \hat{X} = 1$
0	0	$1-q$	0	0	11	$p^2 \rightarrow \hat{X} = 1$
1	1	q	1	1	00	$p^2 \rightarrow \hat{X} = 0$
1	1	q	1	1	01	$p(1-p) \rightarrow \hat{X} = 1$
1	1	q	1	1	10	$p(1-p) \rightarrow \hat{X} = 1$
1	1	q	1	1	11	$(1-p)^2 \rightarrow \hat{X} = 1$

$$P_{a_{ave}}^{(e)} = \frac{\Pr(Y \neq Xf(X))}{2} = \frac{1}{2}(1-q)(2p(1-p) + p^2) + \frac{1}{2}q(p^2)$$

$$= p - \frac{p^2}{2} + q(-p + p^2) < p/2.$$

This slide applies MAP decision rules, exploiting the prior bias $q > 1/2$, to the two-copy repetition code. This page explains how priors influence MAP decisions and presents the resulting average error probability expression. Emphasize the point: decoding performance depends on both channel noise and source statistics; MAP properly uses priors to improve decisions when available.

Slide: MAP decoding (repetition), case $q < 1/2$

MAP decoding : single-bit+one-copy (repetition code):
send $(X, f(X))$, receive two-bit Y with independent BSC
flips.

Assume that $q < 1/2$

X	$f(X)$	$\theta_{Xf(X)}$	X	$f(X)$	Y	$\theta_{Y Xf(X)}$
0	0	$1-q$	0	0	00	$(1-p)^2 \rightarrow \hat{X} = 0$
0	0	$1-q$	0	0	01	$p(1-p) \rightarrow \hat{X} = 0$
0	0	$1-q$	0	0	10	$p(1-p) \rightarrow \hat{X} = 0$
0	0	$1-q$	0	0	11	$p^2 \rightarrow \hat{X} = 1$
1	1	q	1	1	00	$p^2 \rightarrow \hat{X} = 0$
1	1	q	1	1	01	$p(1-p) \rightarrow \hat{X} = 0$
1	1	q	1	1	10	$p(1-p) \rightarrow \hat{X} = 0$
1	1	q	1	1	11	$(1-p)^2 \rightarrow \hat{X} = 1$

$$P_{a_{ave}}^{(e)} = \frac{\Pr(Y \neq Xf(X))}{2} = \frac{1}{2}(1-q)(p^2) + \frac{1}{2}q(2p(1-p) + p^2)$$

$$= \frac{p^2}{2} + q(p - p^2) < p/2.$$

This complementary slide shows the other prior regime $q < 1/2$ and how MAP rules change. The short takeaway for this part is: when the prior flips sign, some received patterns that previously pointed to one bit now point to the other; quantitatively, the average error probability has a different expression. This is a small concrete example of the general principle that prior information can be as important as channel observations.

Slide: Average error probability (MAP) — compact expressions

Average error probability (MAP) — compact expressions.

$$P_{ave}^{(e)} = \begin{cases} p, & \text{without any Parity,} \\ p - \frac{p^2}{2}, & \text{1bit Parity, No MAP Decoding,} \\ p - \frac{p^2}{2} - q(p - p^2) < p/2, & \text{1bit Parity, MAP Decoding, } q > 1/2, \\ \frac{p^2}{2} + q(p - p^2) < p/2, & \text{1bit Parity, MAP Decoding, } q < 1/2. \end{cases}$$

- Single-bit transmission (no redundancy): if the channel flips the bit, receiver has no way to tell.
- Redundancy gives the decoder additional evidence; even if some symbols flip, the overall message can still be recovered.
- But redundancy costs rate — designing codes trades redundancy vs reliability, without parity $R = 1$, with parity $R = 1/2$.

Navigation icons: back, forward, search, etc.

Hassan Tavakoli

Graph-Based Error Correcting Code

12 / 44

This slide summarizes several closed-form error probability expressions: no parity, parity without MAP, parity with MAP (two prior regimes). Here, we restate these expressions and discuss their meaning: parity (redundancy) reduces error but also halves rate in the toy example; MAP decoding extracts the most value from redundancy by using priors. Note also practical implications: for real codes we rarely have closed forms, motivating simulation and asymptotic analysis (density evolution).

Slide: Graphical Thinking (first)

Graphical Thnking

X	$f(X)$	$\theta_{Xf(X)}$
0	0	$1 - q$
1	1	q

constraint: $C = X \oplus f(X) = 0$

Navigation icons

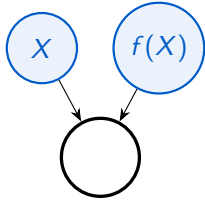
Hassan TavakoliGraph-Based Error Correcting Code13 / 44

This slide begins the shift to factor/Tanner graphs by showing a simple XOR constraint connecting a variable and its copy. Here, we explain how parity constraints are local factors in a bipartite graph and how that representation is convenient for message passing: each factor enforces a local algebraic condition (e.g., $X \oplus f(X) = 0$) and messages transmit local beliefs along edges.

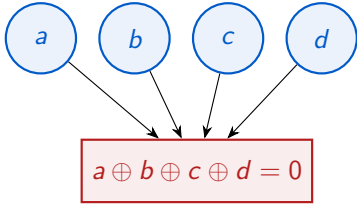
Slide: Graphical Thinking (second)

Graphical Thinking

X	P	θ_{XP}
0	0	$1 - q$
1	1	q



constraint: $C = X \oplus f(X) = 0$



enforces $a \oplus b \oplus c \oplus d = 0$

Hassan Tavakoli

Graph-Based Error Correcting Code

14 / 44

This follow-up slide shows a larger parity factor connecting four variables and emphasizes how one factor can enforce a global parity of several bits. It should be noted that Tanner graphs represent parity-check matrices H : each check node corresponds to a row of H , connecting to the variable nodes corresponding to nonzero entries. This directly motivates LDPC codes, where H is sparse and the graph has low-degree nodes.

Slide: Graphical encoding for Hamming(7,4)

Graphical encoding for Hamming(7,4)

Standard Hamming(7,4) layout:

Parity equations (bits in positions shown above):

$$Pa_1 = d_1 \oplus d_2 \oplus d_4 \quad (\text{positions } 3, 5, 7)$$
$$Pa_2 = d_1 \oplus d_3 \oplus d_4 \quad (\text{positions } 3, 6, 7)$$
$$Pa_3 = d_2 \oplus d_3 \oplus d_4 \quad (\text{positions } 5, 6, 7)$$

Numeric example: $m = [d_1, d_2, d_3, d_4] = [1, 0, 1, 1]$

Compute:

$$Pa_1 = 1 \oplus 0 \oplus 1 = 0 \quad Pa_2 = 1 \oplus 1 \oplus 1 = 1 \quad Pa_3 = 0 \oplus 1 \oplus 1 = 0$$

Encoded codeword: $c = [0, 1, 1, 0, 0, 1, 1]$

Hassan TavakoliGraph-Based Error Correcting Code15 / 44

This slide draws the Tanner/factor graph for Hamming(7,4) and provides parity equations along with a numeric example. We explain how the three parity bits are linear combinations of the four data bits (positions and equations given), how the parity-check matrix H arises, and how the graph supports syndrome-based decoding and single-error correction. Include the numeric example: mapping message $[1, 0, 1, 1]$ to codeword $[0, 1, 1, 0, 0, 1, 1]$ demonstrates concrete encoding.

Slide: Graph-Based-Codes-1

Graph-Based-Codes-1

- **Graphical representation.** Codes are described by a Tanner / factor graph with variable (bit) nodes and check (parity) nodes — this gives an intuitive, local view of constraints.
- **Rate (linear codes).** For length n , parity-check matrix H with m checks, the code rate is

$$R = 1 - \frac{\text{rank}(H)}{n} \approx 1 - \frac{m}{n},$$

(exact value depends on independent checks).

- **Sparsity (LDPC).** Parity-check matrix is sparse: average node degrees are $O(1)$, which reduces complexity and memory.
- **Message-passing decoding.** Belief propagation (BP) / sum-product (or min-sum) uses only local messages; decoding is iterative and highly parallelizable.
- **Complexity.** Per iteration complexity is $O(n)$ (proportional to number of edges); total cost $\approx O(n \times \text{\#iterations})$.

This slide summarizes the major properties of graph-based codes: Tanner graphs, rate formula $R \approx 1 - m/n$, sparsity of LDPC parity-check matrices, message-passing decoding, and per-iteration complexity $O(n)$. We provide more formal detail on the rank vs. number of checks distinction, and note that sparsity is what makes LDPC practical (low memory and localized computation).

Slide: Graph-Based-Codes-2

Graph-Based-Codes-2

- **Threshold phenomenon.** Ensembles have a noise threshold (found by density evolution): below it BP decoding succeeds with high probability as $n \rightarrow \infty$.
- **Regular vs. irregular.** Irregular degree distributions often yield better thresholds (closer to capacity) than regular graphs.
- **Girth and cycles.** Short cycles (especially 4-cycles) degrade BP convergence and performance; girth design is important.
- **Error-floor vs. waterfall.** Typical BER curve has a steep “waterfall” region and a high-SNR “error floor” caused by small trapping sets / near-codewords.
- **Minimum distance.** Depends on construction — can grow linearly for some ensembles (good ML performance) or sublinearly for others; impacts error floor.

This slide discusses threshold phenomena, regular vs irregular ensembles, girth, error-floor vs waterfall, and minimum distance. We expand: density evolution finds BP thresholds; irregular ensembles optimize degree distributions for better thresholds; small cycles (girth 4) create correlation loops that spoil BP's tree-like approximations; the error floor originates from small trapping sets that do not disappear with iterative decoding.

Slide: Graph-Based-Codes-3

Graph-Based-Codes-3

- **Encoding.** Naïve encoding is $O(n^2)$, but practical constructions (systematic form, lower-triangular structure, or generator from sparse H) allow linear or near-linear encoding.
- **Flexibility and design tools.** Puncturing, shortening, protographs, and degree-distribution optimization are used to tune rate and performance.
- **Hardware friendliness.** Local computation + parallelism → efficient ASIC/FPGA implementations (throughput, low latency).
- **Applications.** Widely used in modern communication and storage standards (satellite, Wi-Fi, 5G, SSDs).

Hassan Tavakoli

Graph-Based Error Correcting Code

18 / 44

Here encoding complexity, flexibility (puncturing, protographs), hardware friendliness, and typical applications are listed. Here, we clarify encoding approaches: systematic and generator-matrix based methods can be optimized to near-linear cost; protographs and lifting provide structured ensembles helpful for implementation; and list standards (e.g., 5G, Wi-Fi, satellite) where LDPC or related graph-based codes are deployed.

Slide: What is a factor graph?

What is a factor graph?

- A **factor graph** is a **bipartite graph** representing how a global function factors into a product of local functions (factors).
- Two node types:
 - **Variable nodes** (usually circles) — each represents a variable x_i .
 - **Factor nodes** (usually squares) — each represents a local function $f_a(\cdot)$ that depends on a subset of variables.
- Edges connect a factor node to each variable it depends on. The graph explicitly shows locality of interactions and is the natural domain for message-passing (sum-product / belief propagation).
- Example use-cases: probabilistic models (Bayes nets / Markov random fields), codes (Tanner graphs are factor graphs for parity-check constraints), and many combinatorial optimization problems.

This slide defines factor graphs and contrasts variable nodes vs factor nodes. We formalize the representation $p(x) = \prod_a f_a(x_{N(a)})$, explain bipartiteness and how message passing computes marginals or MAP, and emphasize that Tanner graphs for linear codes are a specific instance where each factor enforces a parity constraint (an indicator function of a linear equation).

Slide: Example factor graph

Example factor graph

• variable
□ factor

```
graph TD; x1((x1)) --- fc[f_c]; x1 --- fa[f_a]; x2((x2)) --- fa; x2 --- fb[f_b]; x3((x3)) --- fb; x4((x4)) --- fb;
```

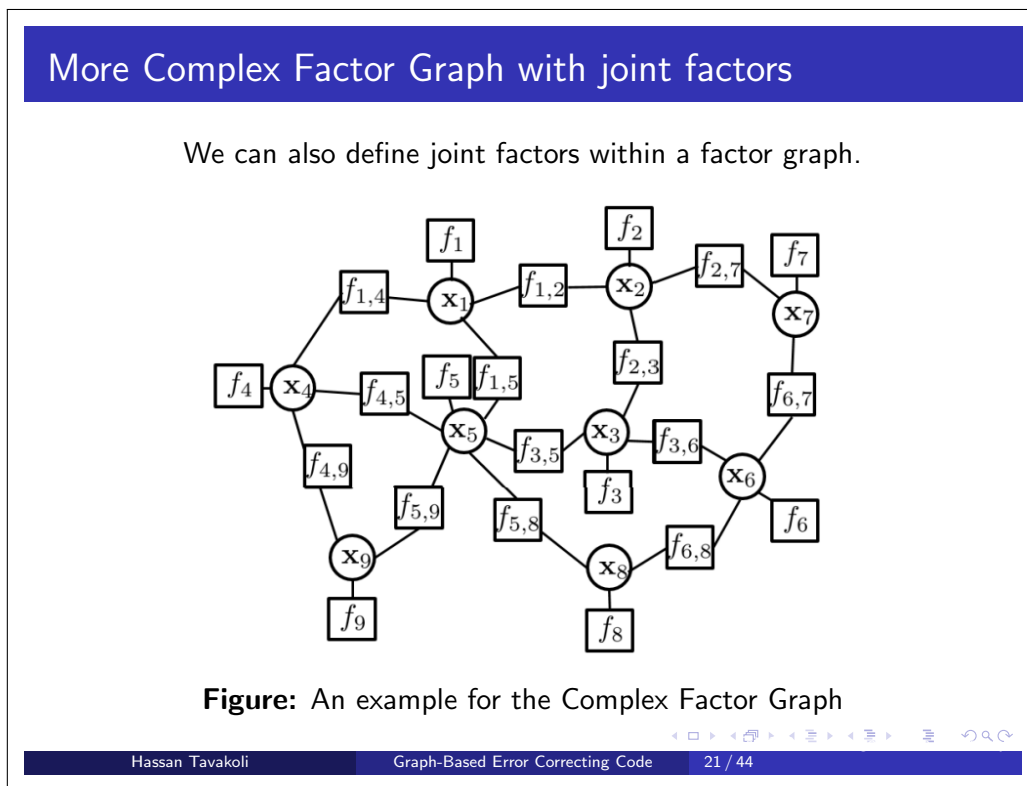
$$p(x_1, x_2, x_3, x_4) = f_a(x_1, x_2) f_b(x_2, x_3, x_4) f_c(x_1)$$

- In this graph f_a connects to x_1, x_2 ; f_b to x_2, x_3, x_4 ; f_c to x_1 .
- Message-passing proceeds by sending messages along edges between variables and factors (sum-product for marginals, max-product for MAP).

Hassan Tavakoli Graph-Based Error Correcting Code 20 / 44

A concrete factor graph with factors f_a, f_b, f_c is shown; the slide notes the factorization $p(x_1, x_2, x_3, x_4) = f_a(x_1, x_2)f_b(x_2, x_3, x_4)f_c(x_1)$. Now, we walk through message directions and emphasize that factor degrees (arity) govern computational cost of a single factor update (sums over the factor's variables), motivating small-degree factors or factor decomposition where possible.

Slide: More Complex Factor Graph with joint factors



This slide displays a larger example where factors may be joint and involve many variables. Here, we warn practitioners that joint high-arity factors can be computationally expensive for exact sum-product updates, and that approximate or specialized updates (e.g., fast Fourier transform for convolutional factors, or canonical forms) are necessary in practice.

Slide: Message Passing

Message Passing

- General framework for computing functions over a graph by passing “messages” between nodes.
- Applicable to factor graphs, Bayesian networks, Markov random fields, etc.
- Can compute marginals, MAP estimates, counts, or optimization tasks.
- Flexible: messages can be sums, products, maxes, probabilities depending on the task.
- Key idea: the mechanism of sending/receiving information along edges.

A set of small navigation icons typically found in Beamer presentations, including symbols for back, forward, search, and other slide navigation functions.

Hassan TavakoliGraph-Based Error Correcting Code22 / 44

This slide presents the general idea of message passing: computing global functions by local communication along edges. Here, we distinguish between different tasks (marginalization, MAP, counting/partition function estimation) and note that the update semantics change accordingly (sums/products vs max/product). Also emphasize that correctness is guaranteed on trees; loopy graphs require approximation.

Slide: Belief Propagation (BP)

Belief Propagation (BP)

- A specific instance of message passing aimed at computing (approximate) marginal probabilities.
- Typically applied to factor graphs or Bayesian networks.
- Uses specific message rules:
 - Variable $\rightarrow$ Factor: product of incoming messages from neighboring factors.
 - Factor $\rightarrow$ Variable: sum/integral over factor times incoming messages from neighboring variables.
- Variants:
 - Sum-Product BP: exact marginals on trees, approximate on loopy graphs.
 - Max-Product BP: computes MAP estimates.
- Key idea: message passing with the goal of inferring node marginals.
- All belief propagation is message passing, but not all message passing is belief propagation.
- Think of message passing as the “tool,” and BP as a “recipe” for using that tool to get probabilistic beliefs.

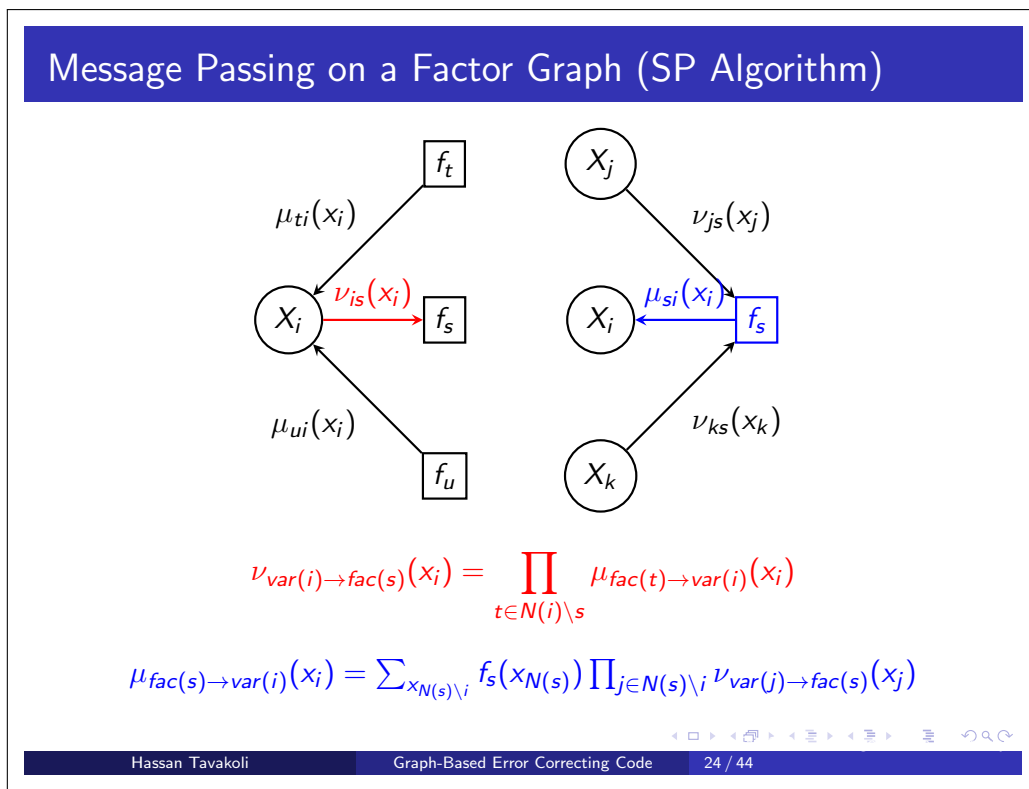
Hassan Tavakoli

Graph-Based Error Correcting Code

23 / 44

This slide frames BP as a recipe for computing (approximate) marginals: variable $\rightarrow$ factor messages multiply incoming factor messages; factor $\rightarrow$ variable messages marginalize the factor times incoming variable messages. We distinguish between sum-product (marginals) and max-product (MAP), note conditions for exactness (graphs without cycles), and mention practical variants (log-domain, normalized messages, min-sum approximation).

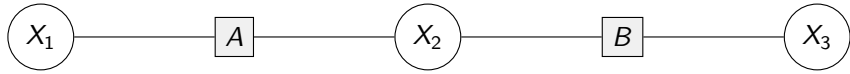
Slide: Message Passing on a Factor Graph (SP Algorithm)



This slide gives the explicit sum-product update equations. We restate these equations, discuss normalization to avoid underflow, and mention implementation choices: synchronous vs asynchronous schedules, damping to improve convergence in loopy graphs, and numerical representations (log-likelihood ratios for binary codes) that speed up computations.

Slide: Example: Graph structure for Assumptions (noisy-equality factors)

Example: Graph structure for Assumptions (noisy-equality factors)



Chain: $X_1 - A(X_1, X_2) - X_2 - B(X_2, X_3) - X_3$.

$X_i \in \{0, 1\}$, $\psi_i(x_i) = P(X_i = x_i)$ (priors)

Factors (noisy equality):

$$\phi_A(x_1, x_2) = \begin{cases} 0.8 & x_1 = x_2, \\ 0.2 & x_1 \neq x_2, \end{cases} \quad \phi_B(x_2, x_3) = \begin{cases} 0.9 & x_2 = x_3, \\ 0.1 & x_2 \neq x_3. \end{cases}$$

Goal: compute message passing and posteriors $b_i(x_i)$.

Hassan Tavakoli Graph-Based Error Correcting Code 25 / 44

This small chain example with noisy equality factors illustrates how non-deterministic constraints (probabilistic factors with preferred equalities) change message calculations. We explain that factors with soft preferences (e.g., $\phi_A(x_1, x_2) = 0.8$ if equal, 0.2 otherwise) allow partial belief propagation rather than a hard XOR, and note how belief updates quantify the tension between noisy factors and priors.

Slide: Case A — Original priors (prior MAP unchanged)-1

Case A — Original priors (prior MAP unchanged)-1

Priors (case A):

$$P(X_1 = 1) = 0.9, \quad P(X_2 = 1) = 0.1, \quad P(X_3 = 1) = 0.1.$$

Thus priors (vectors): $\psi_1 = [0.1, 0.9]$, $\psi_2 = [0.9, 0.1]$, $\psi_3 = [0.9, 0.1]$.

textbf{Leaf} $\rightarrow$ factor (priors):

$$m_{X_1 \rightarrow A} = [0.1, 0.9], \quad m_{X_3 \rightarrow B} = [0.9, 0.1].$$

Factor $A \rightarrow X_2$:

$$\begin{aligned} m_{A \rightarrow X_2}(x_2 = 0) &= \phi_A(x_1 = 0, x_2 = 0) \cdot \psi_1(x_1 = 0) \\ &\quad + \phi_A(x_1 = 1, x_2 = 0) \cdot \psi_1(x_1 = 1), \end{aligned}$$

$$\begin{aligned} m_{A \rightarrow X_2}(x_2 = 1) &= \phi_A(x_1 = 0, x_2 = 1) \cdot \psi_1(x_1 = 0) \\ &\quad + \phi_A(x_1 = 1, x_2 = 1) \cdot \psi_1(x_1 = 1). \end{aligned}$$

$$m_{A \rightarrow X_2}(x_2 = 0) = 0.8 \cdot 0.1 + 0.2 \cdot 0.9 = 0.08 + 0.18 = 0.26,$$

$$m_{A \rightarrow X_2}(x_2 = 1) = 0.2 \cdot 0.1 + 0.8 \cdot 0.9 = 0.02 + 0.72 = 0.74.$$

Navigation icons: back, forward, search, etc.

This slide starts a worked example with priors $\psi_1 = [0.1, 0.9]$, $\psi_2 = [0.9, 0.1]$, $\psi_3 = [0.9, 0.1]$. We show the factor $\rightarrow$ variable computation $m_{A \rightarrow X_2}$ step-by-step (as given) and explain the intuition: the left leaf pushes toward $X_2 = 1$ or 0 through the factor's soft preference, quantified by the numeric messages computed.

Slide: Case A — Original priors (prior MAP unchanged)-2

Case A — Original priors (prior MAP unchanged)-2

Factor $B \rightarrow X_2$:

$$m_{B \rightarrow X_2}(0) = 0.9 \cdot 0.9 + 0.1 \cdot 0.1 = 0.81 + 0.01 = 0.82,$$

$$m_{B \rightarrow X_2}(1) = 0.1 \cdot 0.9 + 0.9 \cdot 0.1 = 0.09 + 0.09 = 0.18.$$

Unnormalized posterior for X_2 :

$$\tilde{b}_2(x) = \psi_2(x) m_{A \rightarrow X_2}(x) m_{B \rightarrow X_2}(x).$$

Numerically:

$$\tilde{b}_2(0) = 0.9 \cdot 0.26 \cdot 0.82 \approx 0.19188, \quad \tilde{b}_2(1) = 0.1 \cdot 0.74 \cdot 0.18 \approx 0.01332.$$

Normalize:

$$b_2(0) \approx 0.935, \quad b_2(1) \approx 0.065.$$

MAP posterior: $\hat{X}_2 = 0$ (no flip).

Hassan Tavakoli

Graph-Based Error Correcting Code

27 / 44

Continuing the calculation, this slide computes $m_{B \rightarrow X_2}$, forms the unnormalized posterior $\tilde{b}_2$, normalizes, and reports $b_2(0) \approx 0.935$. Here, we emphasize how the product of factors and prior yields a strongly skewed posterior — a useful demonstration of how two relatively weak factors combine multiplicatively to produce a strong belief.

Slide: Case B — Adjusted priors (part 1)

Case B — Adjusted priors (prior $\rightarrow$ different)-1

Now change the prior of X_2 to force a prior MAP mismatch:

$$P(X_1 = 1) = 0.9, \quad P(X_2 = 1) = 0.9, \quad P(X_3 = 1) = 0.1.$$

So $\psi_2 = [0.1, 0.9]$ now strongly favors 1.

Factor messages (unchanged):

$$m_{A \rightarrow X_2} = [0.26, 0.74], \quad m_{B \rightarrow X_2} = [0.82, 0.18].$$

Unnormalized posterior for X_2 :

$$\tilde{b}_2(0) = \psi_2(0) m_{A \rightarrow X_2}(0) m_{B \rightarrow X_2}(0) = 0.1 \cdot 0.26 \cdot 0.82 \approx 0.02132,$$

$$\tilde{b}_2(1) = \psi_2(1) m_{A \rightarrow X_2}(1) m_{B \rightarrow X_2}(1) = 0.9 \cdot 0.74 \cdot 0.18 \approx 0.11988.$$

Navigation icons: back, forward, search, etc.

Here the prior of X_2 is changed to favor 1 strongly ($\psi_2 = [0.1, 0.9]$). We explain that factor messages remain identical, but the prior flips the balance, leading to a different posterior — this is a concise demonstration that priors can dominate local evidence when sufficiently strong.

Slide: Case B — Adjusted priors (part 2)

Case B — Adjusted priors (prior → different)-2

Normalize:

$$b_2(0) \approx \frac{0.02132}{0.02132 + 0.11988} \approx 0.151, \quad b_2(1) \approx 0.849.$$

MAP posterior: $\hat{X}_2 = 1$ (posterior MAP matches the strong prior).
⇒ Prior pushed MAP to 1; if we reduce $P(X_2 = 1)$ (e.g. to 0.6) the posterior may flip back to 0.

 Hassan Tavakoli Graph-Based Error Correcting Code 29 / 44

This slide normalizes $\tilde{b}_2$ and finds $b_2(1) \approx 0.849$, concluding that the MAP follows the prior. We note the lesson: inference in graphical models balances prior and factor evidence multiplicatively; thresholds exist for prior mass at which the posterior flips — this leads naturally to the optional threshold slide.

Slide: Comparison and interpretation

Comparison and interpretation

- **Case A (weak prior on X_2):** $\psi_2 = [0.9, 0.1]$ (prefers 0) $\Rightarrow$ posterior $b_2(0) \approx 0.935$. MAP agrees with neighbors: $X_2 = 0$.
- **Case B (strong prior on X_2):** $\psi_2 = [0.1, 0.9]$ (prefers 1) $\Rightarrow$ posterior $b_2(1) \approx 0.849$. MAP follows prior despite factor B pulling toward 0.
- **Practical inference lesson:** If priors are strong they can override local evidence; if priors are moderate, structured factor evidence can change MAP. There is a threshold region (in prior mass) where the MAP flips.

Hassan Tavakoli

Graph-Based Error Correcting Code

30 / 44

The slide summarizes Case A vs Case B and draws practical inference lessons: strong priors override structure, while moderate priors can be overturned. We explain how this motivates sensitivity analyses when priors are uncertain, and discuss possible strategies (empirical Bayes, hierarchical priors) to mitigate overconfident incorrect priors in real systems.

Slide: Optional: threshold condition (when posterior flips)

Optional: threshold condition (when posterior flips)

For X_2 , the posterior ratio is

$$\frac{b_2(1)}{b_2(0)} = \frac{\psi_2(1)}{\psi_2(0)} \cdot \frac{m_{A \rightarrow X_2}(1)}{m_{A \rightarrow X_2}(0)} \cdot \frac{m_{B \rightarrow X_2}(1)}{m_{B \rightarrow X_2}(0)}.$$

The MAP flips from 0 to 1 when this ratio crosses 1. Solve for the prior odds:

$$\frac{\psi_2(1)}{\psi_2(0)} > \left(\frac{m_{A \rightarrow X_2}(0)}{m_{A \rightarrow X_2}(1)} \right) \left(\frac{m_{B \rightarrow X_2}(0)}{m_{B \rightarrow X_2}(1)} \right).$$

Plugging numbers from above gives the concrete threshold for these factor messages.

This analytical slide writes the posterior odds and the condition under which MAP flips, solving for the required prior odds. We expand on the inequality and show how plugging numeric messages yields a concrete threshold for $\psi_2(1)/\psi_2(0)$. This is useful for designing robust decoders or understanding how much bias is needed from a side channel to change the decision.

Slide: Factor Graph structure for Graph-Based Code

Factor Graph structure for Graph-Based Code

```

graph LR
    X1((X1)) --- A[A]
    A --- X2((X2))
    X2 --- B[B]
    B --- X3((X3))
    
```

Chain: $X_1 - A(X_1, X_2) - X_2 - B(X_2, X_3) - X_3$.

$X_i \in \{0, 1\}$, $\psi_i(x_i) = P(X_i = x_i)$ (priors)

Factors as XORs: $X_1 \oplus X_2 = 0, X_2 \oplus X_3 = 0$:

$$\phi_A(x_1, x_2) = \begin{cases} \textcolor{red}{0.8} \rightarrow 1 & x_1 = x_2, \\ \textcolor{red}{0.2} \rightarrow 0 & x_1 \neq x_2, \end{cases} \quad \phi_B(x_2, x_3) = \begin{cases} \textcolor{red}{0.9} \rightarrow 1 & x_2 = x_3, \\ \textcolor{red}{0.1} \rightarrow 0 & x_2 \neq x_3. \end{cases}$$

Hassan Tavakoli
Graph-Based Error Correcting Code
32 / 44

This slide restates the chain example but now with hard XOR factors (indicated by replacing soft weights with deterministic indicator values). We make explicit that deterministic parity-check factors correspond to rows of the binary parity-check matrix H , and that peeling and BP on such graphs are the canonical decoding approaches for sparse linear codes.

Slide: Binary Erasure Channel and Peeling decoder-1

Binary Erasure Channel and Peeling decoder-1

Peeling is a stripped-down, hard decisions only, no probabilities, ultra-simplified version of message passing. It works through deterministic propagation of known symbols. It also works only in certain settings—primarily:

- Binary Erasure Channel (BEC)
- LT/Raptor codes
- Some sparse-graph inference problems

How it works:

- If a check node has only one unknown variable, you can solve it → “peel” it.
- Remove that variable and update neighbors.
- Repeat until nothing is solvable.

Hassan Tavakoli

Graph-Based Error Correcting Code

33 / 44

Introduce the BEC and peeling decoder: an erasure model where symbols are either received or erased, and peeling uses checks with exactly one unknown neighbor to deterministically recover variables. We explain why the BEC is analytically convenient (messages are ternary $\{0, 1, ?\}$), how peeling is a deterministic special-case of message passing, and that ensemble thresholds on the BEC are easier to analyze (density evolution simplifies).

Slide: Binary Erasure Channel and Peeling decoder-2

Binary Erasure Channel and Peeling decoder-2

Prior on the transmitted bit X : $P(X = 1) = q$, $P(X = 0) = 1 - q$. The error probability for Single-bit transmission, send X , receive Y , in Binary Erasure Channel is:

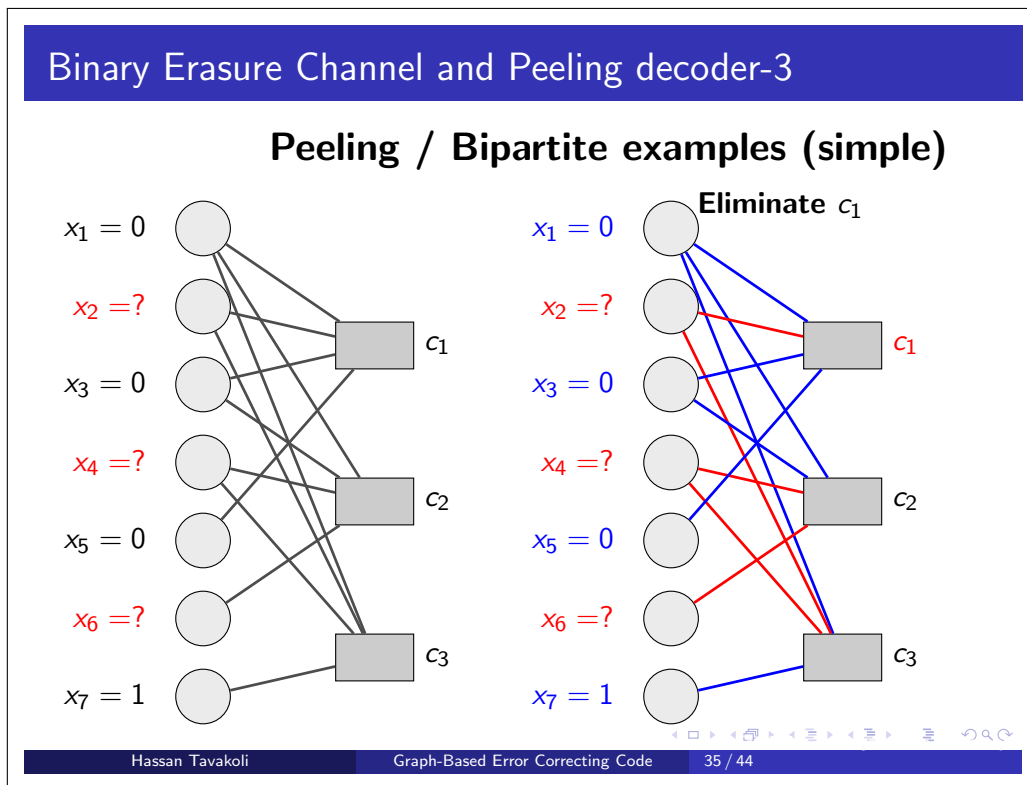
X	θ_x	X	Y	$\theta_{Y X}$
0	$1 - q$	0	0	$1 - p$
0		0	?	p
1	q	0	1	0
		1	0	0
		1	?	p
		1	1	$1 - p$

$$\Pr(Y \neq X) = (1 - q)p + qp = p$$

Hassan Tavakoli
Graph-Based Error Correcting Code
34 / 44

This slide shows the BEC transition matrix and reiterates the erasure probability p . Here, we clarify notation and the difference to BSC: BSC flips bits, BEC erases them (so no ambiguity when symbol is present). Emphasize that many design analyses for LDPC start on the BEC because the peeling process yields exact and tractable recursions.

Slide: Binary Erasure Channel and Peeling decoder-3 (bipartite example)



The slide presents a small bipartite graph and labels some variables as known or erased. We step through the peeling initialization: known variables produce satisfied checks (or remove edges) and checks of degree one reveal unknown variables. Use this example to illustrate the iterative nature of peeling and how local structure determines solvability.

Slide: Binary Erasure Channel and Peeling decoder-4 (decoding step)

Algorithm: Binary Erasure Channel and Peeling decoder-4

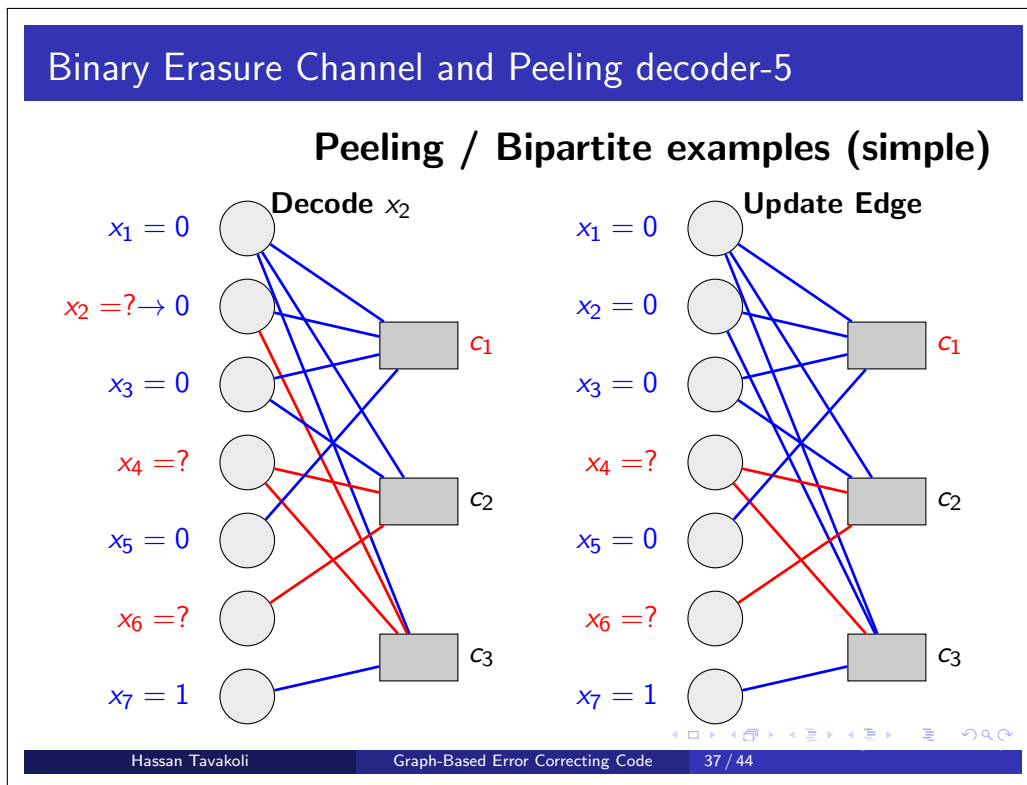
- Decoding step:
 - Select, if possible, a check node with one edge remaining; forward its parity, thereby determining the connected variable node
 - Delete the check node and its outgoing edge
 - Follow the procedure in the initialization process at the known variable node
- Termination
 - If the remaining graph is empty, the codeword is determined
 - If the decoding step gets stuck, declare decoding failure



Hassan TavakoliGraph-Based Error Correcting Code36 / 44

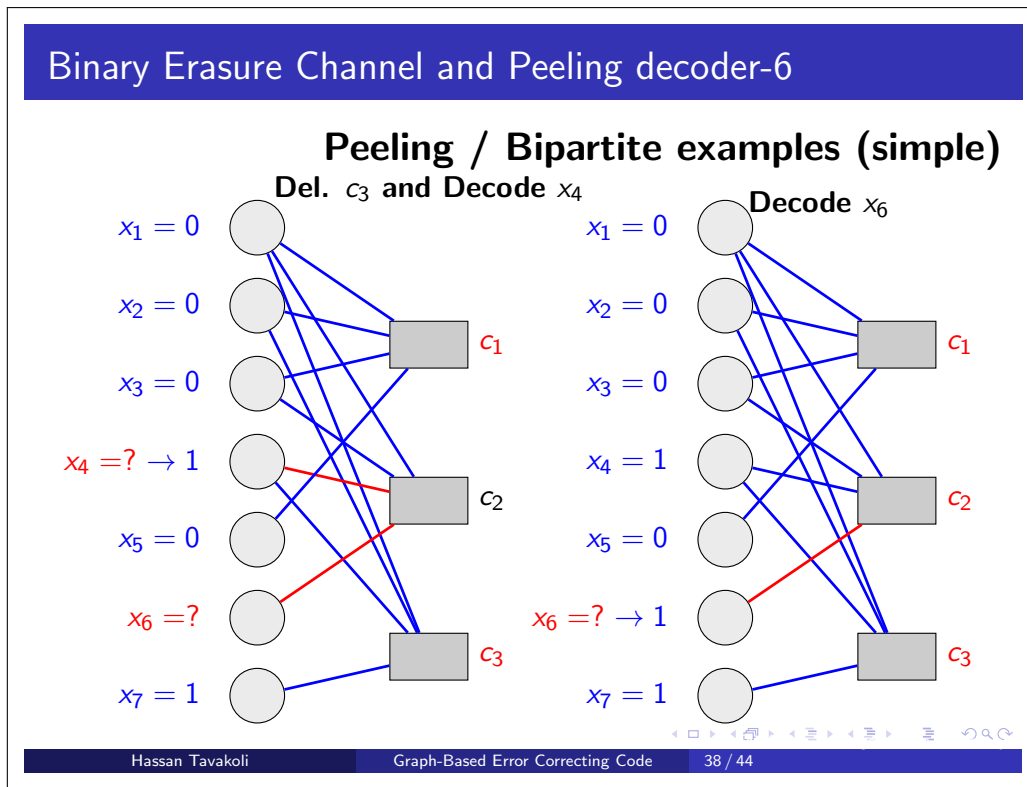
This slide lists the algorithmic steps of peeling and termination conditions. We formalize the algorithm as repeated elimination of degree-1 checks and explain failure modes: if the residual graph has no degree-1 check the decoder gets stuck, corresponding to stopping sets. This connects to ensemble-level thresholds: for successful decoding the residual subgraph must vanish with high probability as $n \rightarrow \infty$.

Slide: Binary Erasure Channel and Peeling decoder-5 (decode x_2 and update)



This slide demonstrates a concrete peeling step: decode x_2 and update neighboring checks/variables. We annotate how edges are removed and how previously high-degree checks may reduce to degree one, cascading recovery. This cascade effect underpins the threshold behavior: below a critical erasure probability, the cascade empties the graph.

Slide: Binary Erasure Channel and Peeling decoder-6 (finish)



Continuing the worked example, this slide shows subsequent deletions and variable decodings (e.g., x_4 , x_6). We highlight how the order of elimination is flexible but the success/failure is determined by the global structure; provide the concept of a stopping set: a set of variable nodes whose neighboring checks are all connected at least twice to that set, causing peeling to stall.

Slide: LDPC Degree Distributions on BEC (Hamming (7,4) as example)

LDPC Degree Distributions on BEC

Hamming (7,4) code

Node degrees: $\Lambda(x) = 3x + 3x^2 + x^3$
 $\Pi(x) = 3x^4$

Number of edges: 12

Edge degrees: $\lambda(x) = \frac{\Lambda(x)}{\Lambda'(1)}, \rho(x) = \frac{\Pi(x)}{\Pi'(1)}$
 $\lambda(x) = \frac{1}{4} + \frac{x}{2} + \frac{x^2}{4}$
 $\rho(x) = x^3$

Rate: $R(\lambda, \rho) = 1 - \frac{\int \rho}{\int \lambda}$
 $R(\lambda, \rho) = 1 - \frac{3}{7} = \frac{4}{7}$

Hassan Tavakoli
Graph-Based Error Correcting Code
39 / 44

This slide computes degree distributions $\Lambda(x)$, $\Pi(x)$, edge-perspective polynomials $\lambda(x)$, $\rho(x)$, number of edges, and rate for Hamming(7,4) as a simple example. We explain the difference between node-perspective (degree counts per node) and edge-perspective distributions used in density evolution, and show how the formula $R = 1 - \int \rho / \int \lambda$ reduces to 4/7 in the example. Stress that these polynomials are the primary description of LDPC ensembles for asymptotic analysis.

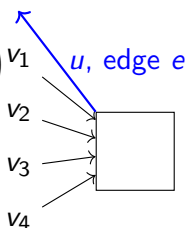
Slide: Message Passing Analysis: Density Evolution-1

Message Passing Analysis: Density Evolution-1

- Consider a check node of degree d with independent incoming messages.

$$\begin{aligned}\Pr(u=?) &= \Pr(v_i=?, \text{ for some } i = 1, \dots, d-1) \\ &= 1 - \Pr(v_i \neq?, \text{ for all } i = 1, \dots, d-1) \\ &= 1 - (1 - p_{\ell-1})^{d-1}\end{aligned}$$

$$\begin{aligned}\Pr(u=?) &= \sum_{d=1}^{d_c} \rho_d \left(1 - (1 - p_{\ell-1})^{d-1} \right) \\ &= 1 - \sum_{d=1}^{d_c} \rho_d (1 - p_{\ell-1})^{d-1} \\ &= 1 - \rho(1 - p_{\ell-1})\end{aligned}$$



Navigation icons: back, forward, search, etc.

Hassan Tavakoli

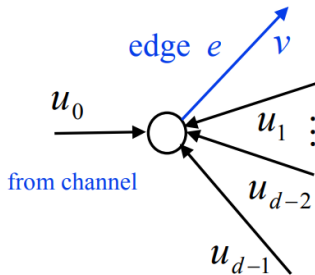
Graph-Based Error Correcting Code

40 / 44

This slide derives the check-to-variable erasure probability update: $\Pr(u=?) = 1 - \rho(1 - p_{\ell-1})$. We derive this expression, explain independence assumptions (cycle-free neighborhood in the large- n limit), and note that density evolution tracks the average message erasure probability p_ℓ across iterations — the central tool for BEC analysis.

Slide: Message Passing Analysis: Density Evolution-2

Message Passing Analysis: Density Evolution-2



- Consider a variable node of degree d with independent incoming messages:

$$\Pr(v = ?) = p_0 [1 - \rho(1 - p_{\ell-1})]^{d-1}$$
- Using the degree distribution λ_d :

$$\Pr(v = ?) = \sum_d \lambda_d p_0 [1 - \rho(1 - p_{\ell-1})]^{d-1} =$$

$p_\ell = p_0 \lambda(1 - \rho(1 - p_{\ell-1}))$

Hassan Tavakoli
Graph-Based Error Correcting Code
41 / 44

Here the variable-to-check update is combined to produce the iteration

$$p_\ell = p_0 \lambda(1 - \rho(1 - p_{\ell-1})).$$

We explain each factor: p_0 is the channel erasure, ρ, λ are degree polynomials, and the fixed point $p_\infty = 0$ stability determines whether decoding succeeds. This slide is the pivot from algorithmic description to ensemble design (threshold computation).

Slide: Optimal Rate for Irregular LDPC Codes in BEC (semi-infinite problem) - 1

Optimal Rate for Irregular LDPC Codes in Binary Erasure Channel Problem: Semi-Infinite Problem!!-1

Optimal rate for irregular LDPC codes in binary erasure channel

Publisher: [IEEE](#) [Cite This](#) [PDF](#) Published in: 2011 IEEE Information Theory Workshop

H. Tavakoli ; M. Ahmadian Attari ; M. Reza Peyghami [All Authors](#)

$$\max_{\{\lambda_{1:d}\}} \sum_{i=1}^d \frac{\lambda_i}{i}$$

subject to:

$$\lambda_i \geq 0, \quad \forall i,$$

$$\sum_i \lambda_i = 1,$$

$$\sum_i \lambda_i [1 - \rho(1 - x)]^{i-1} \leq x, \quad \forall x \in [0, \varepsilon].$$

Hassan Tavakoli
Graph-Based Error Correcting Code
42 / 44

This slide formulates the optimal rate design as a semi-infinite program: maximize rate (equivalently minimize average variable degree) subject to the density evolution inequality holding for all $x \in [0, \varepsilon]$. We explain why the constraint is semi-infinite (infinitely many x values), the objective's relation to rate via harmonic sums, and the practical challenge: infinite constraints require discretization or dualization for numeric solution.

Slide: Optimal Rate for Irregular LDPC Codes in BEC Problem - 2

Optimal Rate for Irregular LDPC Codes in Binary Erasure Channel Problem: Semi-Infinite Problem!!-2

$$\max_{\{\lambda_{1:d}\}} \sum_{i=1}^d \frac{\lambda_i}{i}$$

subject to:

$$\lambda_i \geq 0, \forall i,$$

$$\sum_i \lambda_i = 1,$$

$$\sum_i \lambda_i [1 - \rho(1 - x)]^{i-1} \leq x, \forall x \in [0, \varepsilon].$$

$$\max_{\{\lambda_{1:d}\}} \sum_{i=1}^d \frac{\lambda_i}{i}$$

subject to:

$$\lambda_i \geq 0, \forall i,$$

$$\sum_i \lambda_i = 1,$$

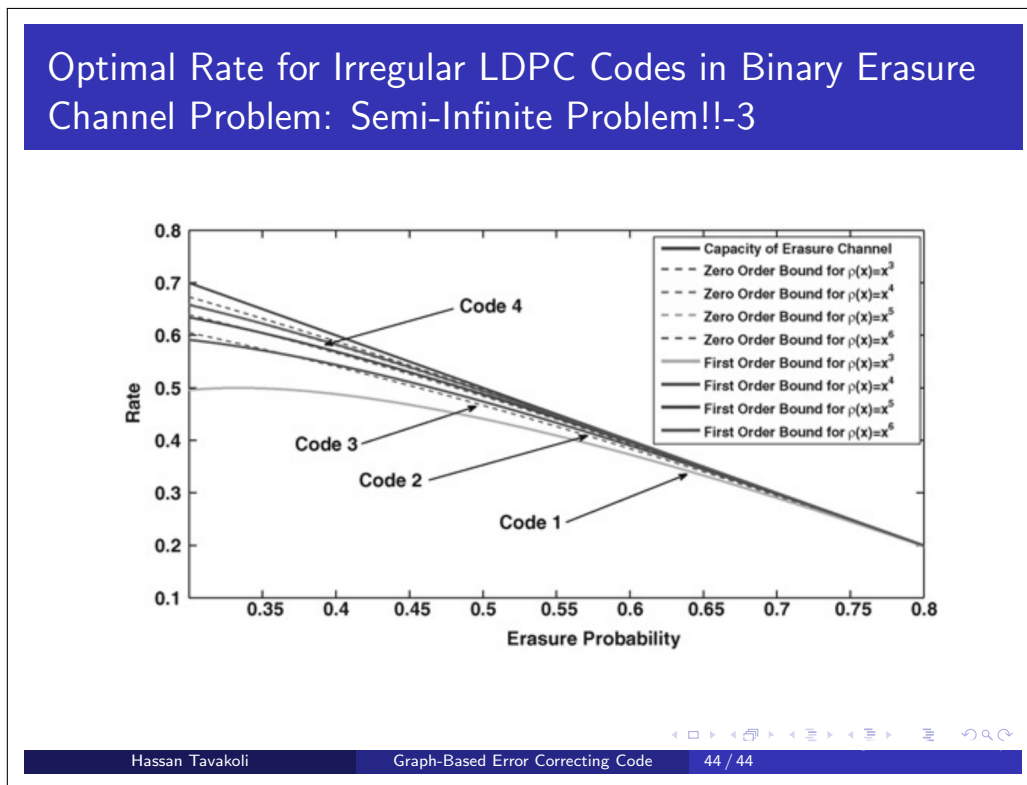
$$A_l = \sum_{i+j=l} B_{ij}, \forall 0 \leq l \leq 2q,$$

$$B_{ij} \geq 0, \quad 0 \leq \lambda_i \leq 1.$$

Hassan Tavakoli
Graph-Based Error Correcting Code
43 / 44

This slide displays two equivalent optimization perspectives and hints at techniques (lifting to moment matrices, linearization via auxiliary variables B_{ij}). We outline computational approaches: discretize x -domain and solve linear program, or derive certificates (sos, Chebyshev approximations) that guarantee constraints over intervals — and point out tradeoffs between tractability and conservatism.

Slide: Optimal Rate for Irregular LDPC Codes in BEC Problem - 3 (rate plot)



This final problem slide shows a rate illustration (likely a plot of achievable rate vs erasure). We interpret the figure: compare designed irregular ensembles to regular ones and to capacity; discuss how close-to-capacity can be achieved with carefully optimized degree distributions and how practical constraints (max degree, error-floor considerations) limit achievable performance.

References

- [1] A. R. Karami, M. A. Attari, and H. Tavakoli, “Multi-layer perceptron neural networks decoder for LDPC codes,” in *Proc. 5th Int. Conf. on Wireless Communications, Networking and Mobile Computing*, 2009.
- [2] H. Tavakoli, M. A. Attari, and M. R. Peyghami, “Optimal rate for irregular LDPC codes in binary erasure channel,” in *Proc. IEEE Information Theory Workshop (ITW)*, 2011, pp. 125–129.
- [3] H. Tavakoli, M. Ahmadian, and M. R. Peyghami, “Optimal rate irregular low-density parity-check codes in binary erasure channel,” *IET Communications*, vol. 6, no. 13, pp. 2000–2006, 2012.
- [4] H. Tavakoli, M. A. Attari, and M. R. Peyghami, “Optimal rate and maximum erasure probability LDPC codes in binary erasure channel,” in *Proc. 49th Annual Allerton Conf. on Communication, Control, and Computing*, 2011.
- [5] H. Tavakoli, “A fast convergence density evolution algorithm for optimal rate LDPC codes in BEC,” in *Proc. 7th International Symposium on Telecommunications (IST)*, Tehran, Iran, Sept. 2014, pp. 574–577, doi: 10.1109/ISTEL.2014.7000770.
- [6] H. Tavakoli, “Source and channel optimal rate LDPC code design for one sender in BE-MAC with source correlation,” in *Proc. 7th International Symposium on Telecommunications (IST)*, Tehran, Iran, Sept. 2014, pp. 578–581, doi: 10.1109/ISTEL.2014.7000771.
- [7] H. Tavakoli, “Reducing the complexity of the linear programming decoding,” *arXiv preprint arXiv:1409.4761*, Sep. 2014. doi: 10.48550/arXiv.1409.4761.
- [8] H. Tavakoli and A. Razi, “Optimizing low density parity check code for two parallel erasure links,” in *Proc. Int. Symp. on Information Theory and Its Applications (ISITA)*, 2016.
- [9] H. Tavakoli, “A family of LDPC codes for binary symmetric channel with optimized rate,” in *Proc. Iranian Conf. on Electrical Engineering (ICEE)*, 2017, pp. 1774–1778.
- [10] H. Tavakoli, “Density evolution analysis of reduced complexity low density parity check,” unpublished manuscript.
- [11] H. Tavakoli, M. A. Attari, and M. R. Peyghami, “Optimal rate for irregular LDPC codes in binary erasure channel,” arXiv:1108.1572 [cs.IT], 2011.
- [12] H. Tavakoli, “Overhead minimization of online codes,” *Telecommunication Systems*, vol. 71, no. 2, pp. 205–214, 2019. doi: 10.1007/s11235-018-0524-3.